

John Kinsky Writing Sample Portfolio

Table of Contents

Writing Sample Portfolio Location	1
Example 1 – Blog for Developers	2
Example 2 – GitHub Readme for Sample Application	7
Example 3 – Get Started Guide For MPI Library	13
Example 4 – User Guide for Analyzer	16
Example 5 – C++ Compiler Developer Guide Overview	20

Writing Sample Portfolio Location

This portfolio file is available from

https://github.com/johnkinsky/book/blob/main/writing_samples.pdf.

I've included a link to the original source for each example and I've included some context to explain the reason for the sample and how the writing impacted the developer audience.

My GitHub repo (<https://github.com/johnkinsky/book>) includes more information about the tools and technologies I've used and some I've scripts developed.

Example 1 – Blog for Developers

Source <https://www.intel.com/content/www/us/en/developer/articles/technical/3-ways-to-get-started-with-oneapi-code-samples.html>

Context for this example: I wrote this blog to help developers quickly locate and use the oneAPI code samples available for each release. According to web telemetry (Adobe Analytics) this article was viewed often. For 18 months after the initial release, it had a very low bounce rate, and it was often cited in feedback as being useful. I also launched and curated the sample page on the Intel Developer Zone (<https://www.intel.com/content/www/us/en/developer/tools/oneapi/code-samples.html>) at the same time. Both of these resources drove users to GitHub code samples.

3 Ways to Get Started with oneAPI Code Samples

If you are like most developers, you don't want to wade through descriptions about new features and capabilities written by someone trying to sell you something. Instead, you would like to see code that works and use the working code to help solve specific problems.

Once you've seen the code work, you are willing to spend time reading a description of the code. We've all been there.

Unfortunately, it is not easy to find reliable or useful sample code or good documentation of the examples when you do find it.

Intel is solving that problem by making available **high-quality, ready-to-use code samples** to develop, offload, and optimize multiarchitecture applications. In this blog, I'll introduce them and then offer three ways to easily access them.

Solve Problems with Code Samples

The samples demonstrate how to solve problems in high-performance computing, code and performance optimization, artificial intelligence and machine learning, and scientific or general graphics rendering.

In fact, the samples are intended **to help any developer at any level** understand concepts and techniques for adapting parallel programming methods to heterogeneous programming for CPUs, GPUs, and FPGAs.

As a new developer just starting out with C++, Fortran, and/or Python* or new to these programming domains, you will find getting-started samples and tutorial samples that walk through the basic concepts to help you orient yourself quickly.

As an experienced developer, you will find many samples to ramp on new techniques quickly, particularly if you are attempting to adapt existing code from vendor-specific C++ code to its SYCL*-compliant equivalent.

For example, you'll find:

- Working versions of guided samples that walk you through the process of migrating CUDA code to equivalent SYCL code.
- The end-to-end process of training models and performing inference using optimizations for PyTorch* and TensorFlow* or performance increases on various CPUs and GPUs.
- FPGA reference designs that you can use to implement nearly complete solutions quickly and tutorials to help you understand FPGA programming.
- Over a hundred samples allowing you to run the code inside Jupyter* Notebooks while reading about the code run in the cell.

Regardless of your experience level, you will find a variety of useful samples—all resident in the GitHub repository—with helpful instructions and commented code.

Ready to dive in? Here are three ways to do it.

1: Sample Browsers Plug-Ins

If you have used the [Intel® oneAPI & AI Tools](#), you might already know that Intel offers sample browsers that plug into Microsoft Visual Studio Code* (see the [Visual Studio Marketplace](#) for information), Microsoft Visual Studio*, and Eclipse* for the current versions of those popular Integrated Development Environments (IDEs) for Linux* and Windows*.

Intel also provides a command-line version of the code sample browser. While you can search for and download the samples from within an IDE, that approach doesn't help much if you are trying to find relevant samples quickly.

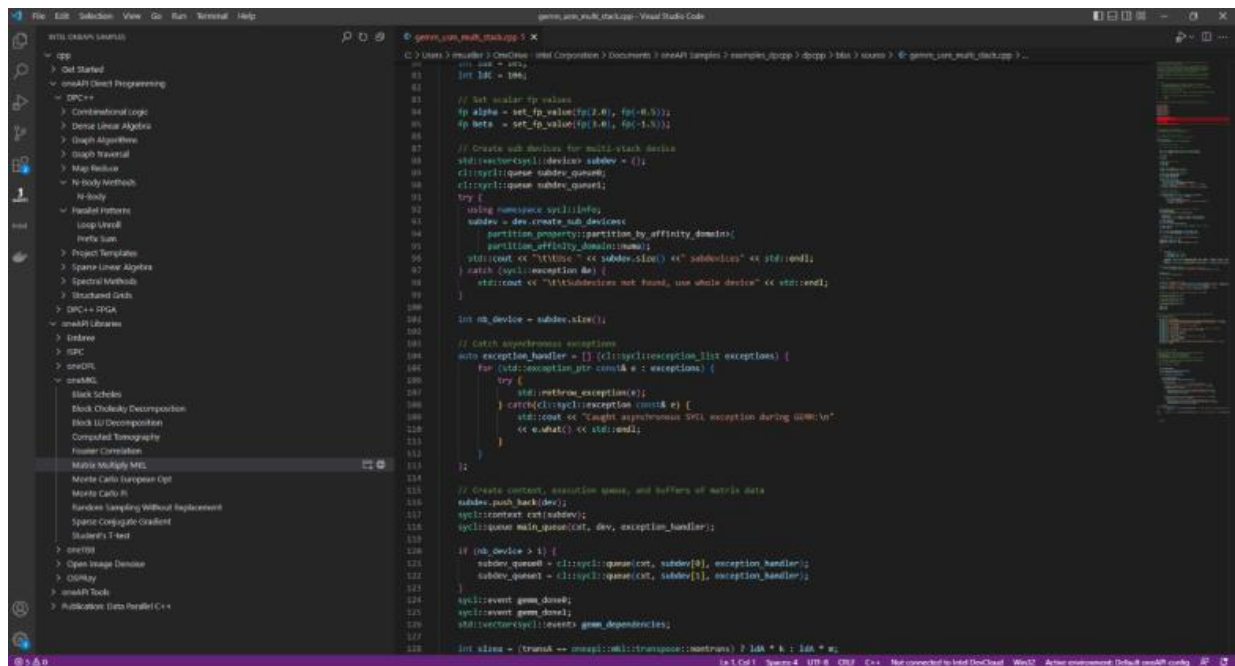


Figure 1. Visual Studio Code oneAPI Samples Browser Plugin

2: Go to GitHub

We have two ways to navigate the samples in GitHub. First, just by browsing the [oneAPI-samples](#) GitHub repository. Second, and for a more dynamic and filterable approach, by visiting the new [Sample Catalog](#) where you can:

- search the sample names and descriptions
- filter by experience level, programming language, or device type to find samples relevant to your CPU, GPU, and FPGA development needs

Since the samples are in GitHub, you can download or clone them directly to your system.

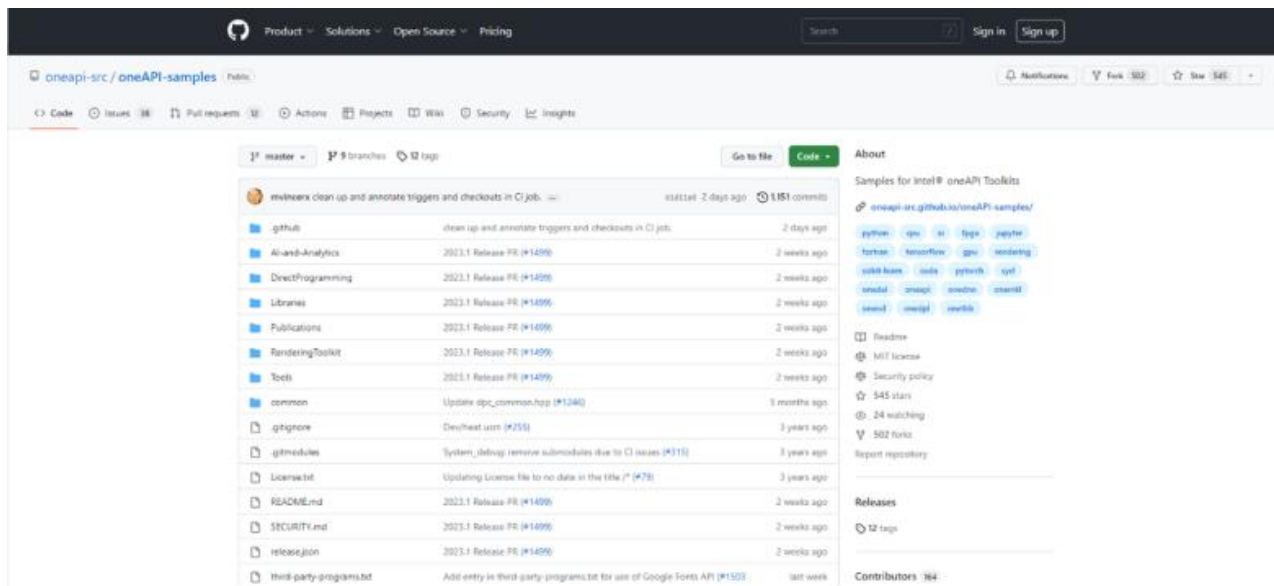


Figure 2. oneAPI-Samples GitHub

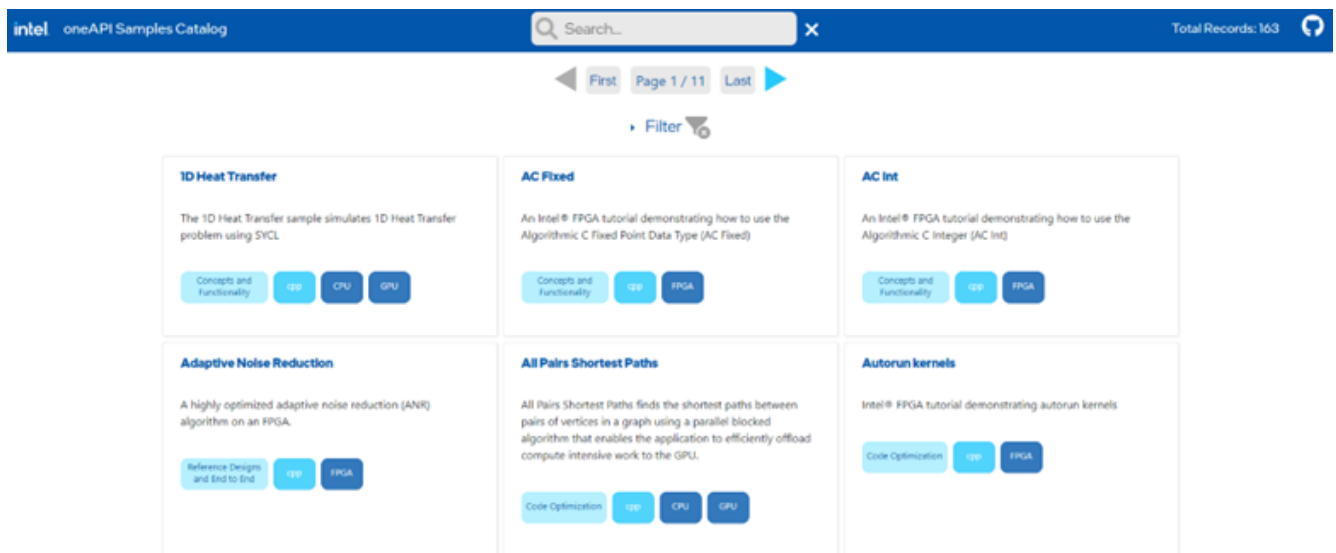


Figure 3. Code Samples Catalog

3: Code Samples Page

As part of a commitment to help developers get to working code as quickly as possible, Intel recently launched a [Code Samples](#) page to highlight samples that might be interesting to developers who are solving problems in ...

- AI and analytics or machine learning
- code or performance optimization for CPUs and GPUs
- rendering and ray tracing

- guidance on how to troubleshoot issues in complex development environments

The new page highlights specific samples in the oneAPI-samples GitHub repository and includes a direct link to the [Samples Catalog](#). You can use the *Code Sample* page or the *Sample Catalog* to find interesting solutions to your development problems.

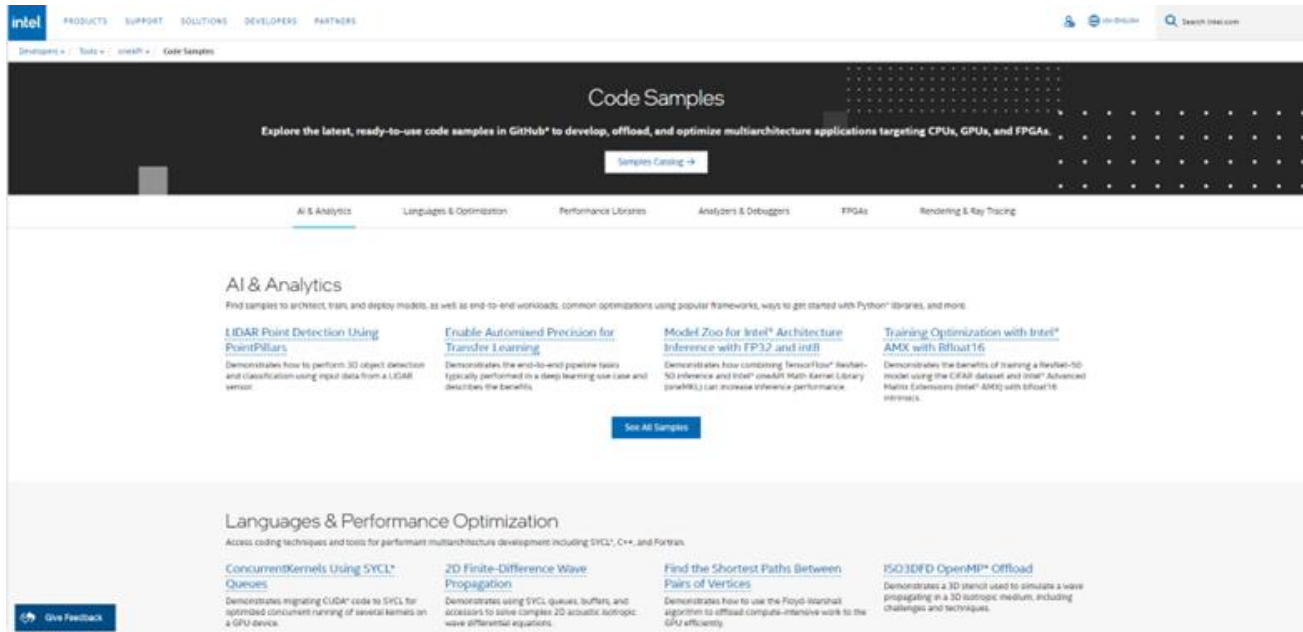


Figure 4. Code Samples Overview Page

Browse the Samples Repository Today

We want to invite you to check out the oneAPI code samples today if you ...

- are you looking for references to implement a specific use case relevant to your domain
- want to find out how to apply heterogeneous cross-architecture programming approaches like SYCL and OpenMP* more effectively to your workload
- are interested in how to best take advantage of oneAPI components and the oneAPI performance library APIs for the libraries you are using in the context of heterogeneous compute

We have many resources available and are continuing to add more code samples in the future aligned to features and capabilities in future Intel oneAPI and AI tools.

Example 2 – GitHub Readme for Sample Application

Source https://github.com/oneapi-src/oneAPI-samples/blob/main/DirectProgramming/C%2B%2BSYCL/DenseLinearAlgebra/matrix_mul/README.md

Context for this example: This readme is an example of a portion of the sample update work I performed as an software optimization engineer. I restructured the readmes, and rewrote the content for clarity, consistency, and brevity. As part of every sample update, I tested the samples, validated the code, rewrote sample readmes, and in some cases made the code changes myself. I worked closely with the sample owners to coordinate all changes through GitHub pull requests.

We use analytics to help track the effectiveness of the changes by tracking the sample satisfaction scores over two years.

Matrix Multiply Sample

The Matrix Multiply is a simple program that multiplies together two large matrices and verifies the results. This program uses two methods to accomplish the same operations: SYCL* and OpenMP (OMP).

Property	Description
What you will learn	How to offload computations on 2D arrays to a GPU using SYCL* and OpenMP.
Time to complete	15 minutes

Note: This Matrix Multiply sample does not support OpenMP on Windows.

Purpose

The Matrix Multiply sample program includes SYCL*-compliant and OpenMP C++ implementations. Each implementation is contained in an appropriately named file: `matrix_mul_sycl.cpp` and `matrix_mul_omp.cpp`. The separation provides a way to compare existing offload techniques such as OpenMP with SYCL* within a relatively simple sample.

The code will attempt to execute on an available GPU first and fallback to the system CPU if a compatible GPU is not detected. The device used for the compilation is displayed in the output.

Prerequisites

Optimized for	Description
OS	Linux* Ubuntu* 18.04 Windows* 10, 11
Hardware	Skylake with GEN9 or newer
Software	Intel® oneAPI DPC++/C++ Compiler Intel® oneAPI C++ Compiler Classic Intel® C++ Compiler

Key implementation details

Since the sample multiplies two large matrices, this sample is a slightly more complex computation than the [Vector Add](#) sample. This sample uses buffers to manage memory. (For more information on different memory management options, refer to the [Vector Add](#) sample.)

The default sample builds the SYCL*-compliant application, and separate OpenMP build instructions are included below.

The code attempts to run the calculation on both the GPU and CPU and verify the results. The size of the computation can be adjusted for heavier workloads. If successful, the name of the offload device and a success message is displayed.

Build the Matrix Multiply Samples

Setting Environment Variables

When working with the Command Line Interface (CLI), you should configure the oneAPI toolkits using environment variables. Set up your CLI environment by sourcing the setvars script every time you open a new terminal window. This practice ensures your compiler, libraries, and tools are ready for development.

Note: If you have not already done so, set up your CLI environment by sourcing the setvars script located in the root of your oneAPI installation.

Linux*:

- For system wide installations: `./opt/intel/oneapi/setvars.sh`
- For private installations: `~/intel/oneapi/setvars.sh`
- For non-POSIX shells, like csh, use the following command: `$ bash -c 'source <install-dir>/setvars.sh ; exec csh'`

Windows*:

- `C:\Program Files (x86)\Intel\oneAPI\setvars.bat`
- For Windows PowerShell*, use the following command: `cmd.exe "/K" ""C:\Program Files (x86)\Intel\oneAPI\setvars.bat" && powershell'`

Microsoft Visual Studio:

- Open a command prompt window and execute `setx SETVARS_CONFIG " "`. This only needs to be set once and will automatically execute the setvars script every time Visual Studio is launched.

For more information on environment variables, see "Use the setvars Script" for [Linux or macOS](#), or [Windows](#).

You can use [Modulefiles scripts](#) to set up your development environment. The modulefiles scripts work with all Linux shells.

If you wish to fine tune the list of components and the version of those components, use a [setvars config file](#) to set up your development environment.

Using Visual Studio Code* (Optional)

You can use Visual Studio Code (VS Code) extensions to set your environment, create launch configurations, and browse and download samples.

The basic steps to build and run a sample using VS Code include:

- Download a sample using the extension **Code Sample Browser for Intel Software Developer Tools**.
- Configure the oneAPI environment with the extension **Environment Configurator for Intel Software Developer Tools**.
- Open a Terminal in VS Code (**Terminal>New Terminal**).
- Run the sample in the VS Code terminal using instructions below.

To learn more about the extensions and how to configure the oneAPI environment, see the [Using Visual Studio Code with Intel® oneAPI Toolkits User Guide](#).

On Linux

Build for SYCL*

1. Build the program.
2. make all

Build for OpenMP

1. Build the program using Make
2. make build_omp

On Windows

Build for SYCL*

Note: The OpenMP offload target is not supported on Windows.

1. Build using nmake.
2. nmake -f Makefile.win build_sycl

On Windows* Using Visual Studio*

- Build the program using VS2017 or later.
 - Right-click on the solution file and open using the IDE.
 - Right-click on the project in **Solution Explorer** and select **Rebuild**.
 - From the top menu, select **Debug > Start without Debugging**.
- Build the program using MSBuild.
 - Open "x64 Native Tools Command Prompt for VS2017" or "x64 Native Tools Command Prompt for VS2019" or whatever is appropriate for your Visual Studio version.
 - Run the following command: MSBuild matrix_mul.sln /t:Rebuild /p:Configuration="release"

Troubleshooting

If you receive an error message, troubleshoot the problem using the **Diagnostics Utility for Intel® oneAPI Toolkits**. The diagnostic utility provides configuration and system checks to help find missing dependencies, permissions errors and other issues. See the [Diagnostics Utility for Intel® oneAPI Toolkits User Guide](#) for more information on using the utility.

Run the Matrix Multiply Samples

On Linux

1. Run both versions.
2. `make run`
3. `make run_omp`

On Windows

1. Run the SYCL version.
2. `nmake -f Makefile.win run_sycl`
3. Alternatively, change directory to the output folder and run the .exe file. (Depending on the build method you selected, the .exe name could be different from what is shown.)
4. `matrix_mul_sycl.exe`

Application Parameters

You can modify the computation size by adjusting the size parameter in the `matrix_mul_sycl.cpp` and `matrix_mul_omp.cpp` files. You can configure the following parameters in both files:

`size = m_size = 150*8; // Must be a multiple of 8.`

`M = m_size / 8;`

`N = m_size / 4;`

`P = m_size / 2;`

Note: The size value must be in multiples of 8.

Example Output

```
./matrix_mul_dpc
```

```
Device: Intel(R) Iris(R) Xe Graphics
```

Problem size: $c(150,600) = a(150,300) * b(300,600)$

Result of matrix multiplication using SYCL: Success - The results are correct!

Example 3 – Get Started Guide For MPI Library

Source <https://www.intel.com/content/www/us/en/docs/mpi-library/get-started-guide-windows/2021-11/overview.html>

Context for this example: This is a single topic from the *Get Started with the Intel® MPI Library for Windows* OS*. (The documentation is separated by operating system due to technical constraints in the publishing platform.)

This topic is an example where I inherited existing content and I restructured and rewrote for clarity assuming a specific audience. At the time, the Intel MPI Library was the largest product Intel released, as measured by install volume, with a dedicated, highly invested developer and engineering base.

The software was available as a stand-alone software package and as part of all oneAPI toolkit installations, so it was widely used by nearly all developers using most of the Intel developer tools.

My changes improved the overall customer satisfaction for these documents as measured by Qualtrics developer survey data.

Get Started with Intel® MPI Library for Intel® oneAPI on Windows* OS

The Intel® MPI Library enables you to create, maintain, and test advanced applications that have performance advantages on high-performance computing (HPC) clusters based on Intel® processors.

The Intel MPI Library is available as a standalone product and as part of the [Intel® HPC Toolkit](#). The Intel MPI Library is a multi-fabric message passing library that implements the Message Passing Interface, version 3.1 (MPI-3.1) specification. Use the library to develop applications that can run on multiple cluster interconnects.

The Intel MPI Library has the following features:

- Low overhead enables analysis of large amounts of data
- MPI tuning utility for accelerating your applications
- Interconnect independence and flexible runtime fabric selection

The product consists of the following main components:

- Compilation tools, including compiler drivers such as mpiicc and mpifort

- Include files and modules
- Dynamic (.dll) libraries, debug libraries, interface libraries, and program database(.pdb) files
- Process Manager and tools to run programs
- Test code
- Documentation provided as a separate package or available from the Intel Developer Zone

Intel MPI Library also includes [Intel® MPI Benchmarks](#), which enable you to measure MPI operations on various cluster architectures and MPI implementations. For details, see the [Intel® MPI Benchmarks User Guide](#). Source code is available in the [GitHub repository](#).

For other more information on using the Intel MPI Library see [Intel® MPI Library Developer Guide for Windows* OS](#) and [Intel® MPI Library Developer Reference for Windows* OS](#).

Key Features

The Intel MPI Library has the following major features:

- MPI-1, MPI-2.2 and MPI-3.1 specification conformance
- Interconnect independence
- Supported Languages:
 - For GNU* compilers: C, C++, Fortran 77, Fortran 95
 - For Intel® compilers: C, C++, Fortran 77, Fortran 90, Fortran 95

Building and Running MPI Programs

Compiling an MPI Program

1. Run the `setvars.bat` script to set the environment variables for the Intel MPI Library. The script is located in the installation directory (by default, `C:\Program Files (x86)\Intel\oneAPI`).

NOTE:

If you are using Intel MPI in a Unified Directory Layout, set the environment variables using the `C:\Program Files (x86)\Intel\oneAPI\<toolkit-version-number>\oneapi_setvars.bat` script instead. To understand more about the Unified Directory Layout, including how the environment is initialized and the advantages of using the layout, see [Use the setvars and oneapi-vars Scripts with Windows*](#).

2. Make sure you have the desired compiler installed and configured properly. For example, for the Intel® C++ Compiler, run:

```
> icl
```

If the command is not recognized, add the compiler to your PATH. For the Intel® compilers, you can run the vars.bat script from corresponding directory.

3. Compile your program using the appropriate compiler driver. For example, for a test C program:

```
> mpiicc -o test.exe <install-dir>\test\test.c
```

For detailed system requirements, see [Intel® MPI Library System Requirements](#).

Running an MPI Program

Execute the program using the `mpiexec` command. For example, for the test program:

```
> mpiexec -n <# of processes> test.exe
```

To specify the hosts to run the program on, use the `-hosts` option:

```
> mpiexec -n <# of processes> -ppn <# of processes per node> -hosts  
<host1>,<host2>,...,<hostN> test.exe
```

Troubleshooting

If you encounter problems when using Intel MPI Library, go through the following general procedures to troubleshoot them:

- Check known issues and limitations in the [Release Notes](#).
- Check hosts accessibility. Run a simple non-MPI application (for example, `hostname utility`) on the problem hosts with `mpiexec`. This check helps you reveal an environmental problem, or connectivity problem (for example, unreachable hosts).
- Run the MPI application with debug information enabled. To enable the debug information, set the environment variable `I_MPI_DEBUG=6`. You can also set a different debug level to get more detailed information. This action helps you find the problem component.

See more details in the “Troubleshooting” section of the [Developer Guide](#).

Example 4 – User Guide for Analyzer

Source <https://www.intel.com/content/www/us/en/docs/gpa/user-guide/2024-3/graphics-trace-analyzer-window-open-trace-capture.html>

Context for this example: This is an example topic from the *Intel® Graphics Performance Analyzers User Guide*.

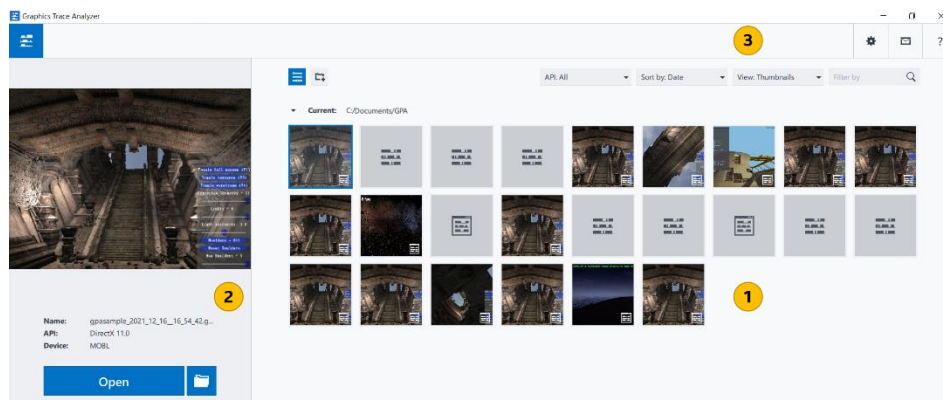
This is another example where I inherited the documentation set that had been started by product engineers and had occasionally been supported by other technical writers. I removed outdated language, updated the overall structure, and made changes consistent with new features. I downloaded and tested the prerelease software to ensure content accuracy.

I was able to allocate about 25% of my time to the project. It became obvious the time I could allocate was not sufficient for the product team to meet their goals, so I suggested migrating the source topics from DITA XML to reStructured Text (rST) to allow engineering-directed publishing. I migrated multiple documents, comprised several hundred topics, from XML to rST, created makefile build scripts, and documented the process of building the new documents using Sphinx. The changes helped engineering own the content changes. Technical writers reviewed and approved the changes. The automated build framework allowed engineering and product teams to quickly release updates and reduce publishing times.

Open Trace Capture

Use the **Open Trace Capture** window to select and open your trace file.

To access the **Open Trace Capture** window, in the Intel® GPA context menu, select **Graphics Trace Analyzer**. On Windows* OS, if you opted to enable context menu entries during installation, you can also open trace files using the Windows context menu.



1

Trace Capture Thumbnails pane

Select the trace capture you want to debug. By default, this field lists all trace captures located in your Intel GPA home directory: %USERPROFILE%\Documents\GPA (Windows* OS). You can sort the trace captures by name or date with the **Sort by** button. If there are no trace captures on your system, these toggle buttons are hidden. The currently selected trace capture is highlighted with blue borders and appears in the trace preview pane.

Trace file thumbnails are marked with the  icon. If the captured trace data is invalid, the thumbnail is marked with the  icon.

To delete captured traces, hover the mouse pointer over a trace thumbnail and click the **Close** button in the top right-hand corner of the thumbnail. Confirm your choice by clicking **Yes** on the thumbnail.

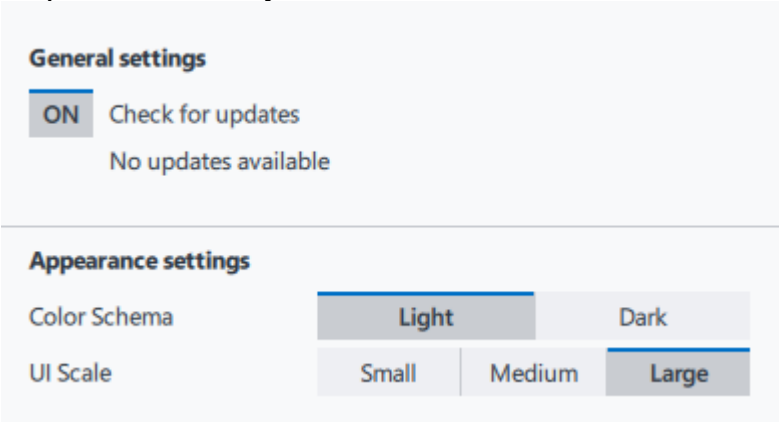
2

Preview pane

Preview the selected trace captures. This field shows the buffer image and the following trace capture properties:

- File name
- API used
- Name of the device on which the trace was captured

To rename the selected trace capture, you can click its filename and enter a new value. Click **Open**, to open the selected trace in the **Platform View**. Click the  **Show in Folder** button to open the storage directory of the selected trace.

3	Message pane View the Intel GPA messages or notifications.
Type filter expression field	Filter the displayed trace capture files by typing in a part of the trace file name.
 Run Graphics Monitor button	Click to capture new traces with Graphics Monitor.
 Settings button	Manage Graphics Trace Analyzer  The screenshot shows the 'General settings' section with a toggle switch for 'Check for updates' set to 'ON' and the text 'No updates available'. Below this is the 'Appearance settings' section, which includes 'Color Schema' with 'Light' and 'Dark' options, and 'UI Scale' with 'Small', 'Medium', and 'Large' options. settings: If a newer version of the product is available, the Settings button is marked with the exclamation mark. To download the update, click the Settings button and select the available version from the context menu. - Use the General Settings toggle button to enable/disable checking for the newest available version of Graphics Trace Analyzer. NOTE: During this check, Graphics Trace Analyzer collects information about your host system. No personally identifiable information is collected. - Use the Color scheme setting to change the palette of the Graphics Trace Analyzer interface from Light (default) to Dark.

	• Use the UI Scale setting to change the size of the Graphics Trace Analyzer screen and the screen elements. Large size is default.
? Help button	Open Intel GPA online documentation.

Example 5 – C++ Compiler Developer Guide Overview

Source: <https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2025-2/>

Context for this example: I owned the C++ Compiler Developer Guide for multiple releases. In response to consistent feedback about the difficulty of finding new information or changed sections in the online version, I updated the overview page for online users to allow developers to quickly find new information and critical sections. The simple change improved feedback scores and resulted in increased initial and return visits/sessions (according to Adobe Analytics tracking data).

Overview

This guide provides information about the Intel® oneAPI DPC++/C++ Compiler and runtime environment. This document is valid for version 2025.2 of the compilers.

The Intel® oneAPI DPC++/C++ Compiler is available as part of the [Intel® oneAPI Base Toolkit](#), [Intel® oneAPI HPC Toolkit](#), [Intel® oneAPI IoT Toolkit](#), or as a [standalone compiler](#).

Refer to the [Intel® oneAPI DPC++/C++ Compiler](#) product page and the [Release Notes](#) for more information about features, specifications, and downloads.

Key Features

The compiler supports these key features:

- **Intel® oneAPI Level Zero:** The Intel® oneAPI Level Zero (Level Zero) Application Programming Interface (API) provides direct-to-metal interfaces to offload accelerator devices.
- **OpenMP* Support:** Compiler support for OpenMP 5.0 Version TR4 features and some OpenMP Version 5.1 features.
- **Pragmas:** Information about directives to provide the compiler with instructions for specific tasks, including splitting large loops into smaller ones, enabling or disabling optimization for code, or offloading computation to the target.
- **Offload Support:** Information about SYCL*, OpenMP, and parallel processing options you can use to affect optimization, code generation, and more.
- **Latest Standards:** Use the latest standards including C++ 20, SYCL, and OpenMP 5.0 and 5.1 for GPU offload.

For more information, refer to the [Intel® oneAPI DPC++/C++ Compiler Introduction](#).

For information about Intel intrinsics, visit [Intel® Intrinsic Guide](#).

Key Sections

Visit these key sections for more information on the compiler:

- **Introduction:** Information on the compiler, including: feature requirements, support, and related information.
- **Compiler Setup:** Information on how to invoke the compiler on the command line or from within an IDE.
- **Compiler Reference:** Information on compiler reference, including: compiler options, compiler limits, libraries, and more.
- **Compilation:** Information about features that can affect compilation, such as environment variables, and using configuration files.
- **Optimization and Programming:** Information about features related to code optimization and program performance improvement.
- **Compatibility and Portability:** Information about conformance to language standards, language compatibility, and portability.

For more information, refer to [Intel® oneAPI DPC++/C++ Compiler Introduction](#).

Clang Option Support

Clang compiler options are supported for this compiler. We do not document these options, but you can check `-help` on the command line to see if a particular option is supported. For more information about Clang options, see the [Clang documentation](#).

Notices and Important Information

- The `dpcpp` driver is deprecated and will be removed in a future release. For SYCL compilation, use the `-fsycl` option with the C++ driver.
- In this document, you may see features labeled as experimental. An experimental feature is one that requires further testing and possible refinement. Depending on testing results, such features may be fully defined and implemented or they may be removed in a future release.
- The Intel® oneAPI DPC++/C++ Compiler does not support macOS*. For macOS or Xcode* support use Intel® C++ Compiler Classic. For more information, visit the [Intel® C++ Compiler Classic Developer Guide and Reference](#).

Use the Compiler: Additional Resources

- **Context Sensitive/F1 Help:** To use the Context Sensitive/F1 Help feature, visit the [Download Documentation: Intel® Compiler \(Current and Previous\)](#) page and follow the provided instructions.

- **Previous Versions of the Developer Guide and Reference:** Visit the [Download Documentation: Intel® Compiler \(Current and Previous\)](#) page to download PDF or FARHTML versions of previous compiler documentation.

When searching HTML files, use a Google Chrome* browser to view your downloaded copy of the Developer Guide and Reference. If you use Mozilla Firefox*, you may encounter an issue where the **Search** tab does not work. As a workaround, you can use the **Contents** and **Index** tabs or a third-party search tool to find your content.